Rapport

application web de quiz

module programmation web.

Ecole Supérieure de Technologie d'Essaouira

Université Cadi Ayyad

2025 - 2026

Encadrant :

Pr. Inass Hasnaoui

Réealisé par :

Salma Jabir
Fatima Ezzahra Labhalla
Hakima Janah
Maryam chaibi
Nouhaila El Hayel
Douha El ouardi

sommaire

- Sommaire
- Introduction
- Description du projet
- Architecture du projet
- Choix techniques
- Difficultés rencontrées
- Répartition de travail
- Interfaces de l'application
- Conclusion

1
Introduction

Ce rapport présente le projet QuickQuiz, une application web de quiz développée dans le cadre

du module de programmation web. L'objectif principal de ce projet est de consolider les

compétences fondamentales en développement web à travers la conception et l'implémentation

d'une application complète intégrant un backend Flask, des templates Jinja2, une base de

données SQLite via SQLAlchemy, et une interface utilisateur en HTML/CSS.

2
Description du projet

QuickQuiz est une plateforme de quiz interactive qui permet aux utilisateurs de tester leurs

connaissances en développement web. L'application propose 5 catégories de quiz : HTML, CSS,

Flask, Jinja2 et SQLAlchemy, avec 10 questions par catégorie.

Fonctionnalités principales :

- Authentification : inscription et connexion sécurisée avec gestion de sessions
- Dashboard personnalisé affichant le username, l'email et le score total
- 5 catégories de quiz avec 10 questions chacune (questions à choix multiples)
- Système de score sur 20 points (2 points par bonne réponse)
- Statut Pass (score >= 10) ou Fail (score < 10)
- Timer de 5 minutes par quiz avec soumission automatique
- Limitation des tentatives : 3 maximum par catégorie

- Interface moderne avec thème sombre et couleurs violettes

3

Architecture du projet

La structure principaledes fichiers est comme cele :

```
/QuickQuiz
    app.py — Backend Flask (routes, logique)
    questions.py — Script d'initialisation de la base de données
    static/
        style.css — Feuille de styles CSS
    templates/
        login.html — Page de connexion
        register.html — Page d'inscription
        dashboard.html — Tableau de bord utilisateur
        quiz.html — Page de quiz
        result.html — Page de résultat
    instance/
        quiz.db — Base de données SQLite
```

Modèles de données :

L'application utilise 3 modèles SQLAlchemy :

- User : id, username, email, password, score
- Question : id, category, question, choice1, choice2, choice3, answer

- Attempt : id, user_id, category, coun

4
Choix techniques

Technologie	Justification
Flask (Python)	Framework léger, adapté aux petites applications web
Jinja2	Moteur de template intégré à Flask, permet l'affichage dynamique
SQLAlchemy	ORM puissant pour gérer la base de données SQLite
SQLite	Base de données légère, ne nécessite pas de serveur
HTML/CSS	Technologies standard pour l'interface utilisateur
Google Fonts (Poppins)	Police moderne et lisible pour le design

5
Difficultés rencontrées

• Manipulation des routes Flask :

La gestion des routes dynamiques (comme /quiz/) a nécessité une compréhension approfondie

du système de routage de Flask, notamment pour passer les paramètres correctement et gérer

les méthodes GET et POST.

• Templates Jinja2 :

L'utilisation correcte des variables dans les templates Jinja2 a posé quelques difficultés,

notamment l'accès aux attributs des objets (user.username au lieu de username) et la syntaxe

des conditions et boucles.

• Questions répétées dans le quiz :

Le script questions.py a été exécuté plusieurs fois, ce qui a causé la duplication des questions

dans la base de données. Ce problème a été résolu en ajoutant une vérification if

Question.query.count() == 0 avant d'insérer les questions.

• Utilisation de Git/GitHub :

Étant la première expérience avec Git et GitHub pour certains membres, la configuration initiale

et la compréhension du workflow (commit, push) ont nécessité un apprentissage rapide.

• Répartition du travail :

Coordonner le travail entre 6 membres a nécessité une bonne organisation pour éviter les

conflits dans le code et assurer une intégration cohérente des différentes parties du projet.

6

Répartition du travail

Membre	Tâches
Fatima Ezzahra Labhalla	Questions du quiz / Tests et débogage / Vidéo de démonstration
Salma Jabir	Frontend (HTML/CSS) / Design de l'interface
Hakima Janah	Backend (Flask/routes) / Rapport
Maryam Chaibi	Base de données / Modèles SQLAlchemy
Nouhaila El Hayel	Base de données SQLAlchemy / Questions
Douha El Ouardi	Backend (Flask/routes) / Logique du quiz

7

Interfaces de l’application

L'application QuickQuiz dispose des interfaces suivantes :

- Page Login : formulaire de connexion avec username et password
- Page Register : formulaire d'inscription avec username, email et password
- Dashboard : affichage du profil utilisateur, score total et 5 catégories de quiz
- Page Quiz : 10 questions à choix multiples avec timer de 5 minutes
- Page Résultat : affichage du score, statut Pass/Fail et bouton Réessayer

8

Conclusion

Le projet QuickQuiz nous a permis de mettre en pratique les compétences acquises durant le

module de programmation web. Nous avons réussi à développer une application web complète

et fonctionnelle intégrant l'authentification, la gestion de sessions, une base de données

relationnelle et une interface utilisateur moderne.

Ce projet nous a également appris à travailler en équipe, à utiliser des outils de versioning

comme Git et GitHub, et à résoudre des problèmes techniques concrets. Les fonctionnalités

bonus (timer, limitation des tentatives, design avancé) ont enrichi l'application et démontrent

notre capacité à aller au-delà des exigences minimales.

GitHub : https://github.com/flabhalla-hub/QuickQuiz